

COMP517 – Data Analysis

Lab 1 –Basic Python Programing

Python is an open source, distributed and freely available language. It has become one of the most adopted and standardized language, offering a comprehensive toolkit for data-driven operations. Python has an extensive collection of libraries, including NumPy, Pandas, SciPy, Scikit-Learn, Matplotlib, Seaborn, and Plotly. These libraries form a full environment for data analysis, widely utilized by data analysts, data scientists, and business analysts.

Objectives The primary goal of this lab is to develop essential skills in data analysis. These skills include the utilization of the Jupyter Notebook and Python built-in arithmetic operators, methods and functions.

In this course we will be using Python3. Various Python IDE (Integrated Development Environment) and code editors such as PyCharm, Spyder, Sublime Text etc, are available. In this course you will be using an open-source web application named jupyter notebook that is maintained by Project Jupyter team. Jupyter's core supported programming languages are Julia, Python, and R. Jupyter ships with the IPython kernel in Jupyter allows you to write your programs in Python. Anaconda is widely recognized as a Python distribution that includes a comprehensive collection of pre-installed packages and tools commonly used in data science and scientific computing. Jupyter Notebook is one of the components that come bundled with Anaconda.

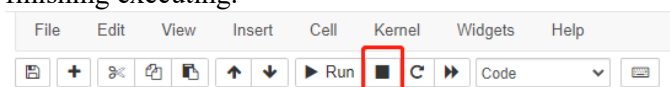
To start with you will need Python3 installed on your PC. To check if the Python installed on your computer open the command line and type Python3:

```
C:\Users\saraz>python3
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun 7 2023, 05:45:37) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
```

If Python3 or Anaconda are not installed on your PC, follow the instructions provided in 'Python Installation and Setup.pdf' file available under Lab1 material.

Tips and Advises:

1. The directory in Jupyter Notebook works just like Windows directory. Any file manipulation from Windows will be reflected on Jupyter Notebook.
2. Do **NOT** try to copy the codes from the examples, type in yourself instead. Otherwise, you might see errors like **SyntaxError**: invalid character ''' (U+2018)
3. You can use `%whos` to view the variables in the kernel. Useful for debugging.
4. You can press the interrupt button next to the run button if for some reason the kernel is not finishing executing.

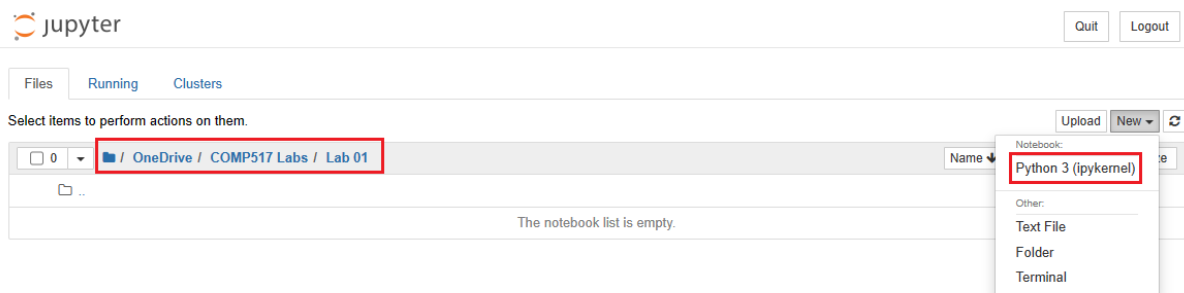


Getting started with Jupyter Notebook

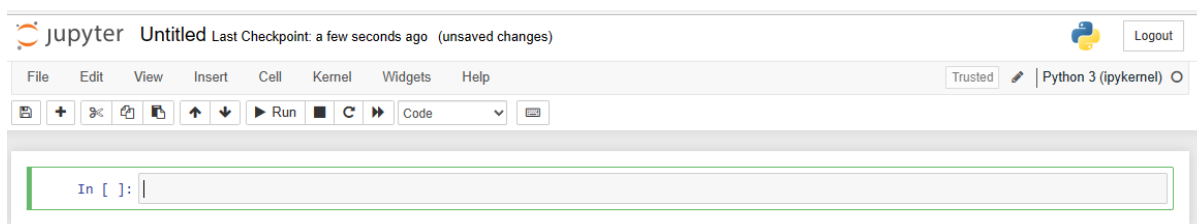
- Run Jupyter via the shortcut Anaconda adds to your start menu. A new tab will open in your default web browser, showing your Notebook Dashboard as below. Note that the default setting

provides access only to the files and sub-folders contained within Jupyter's start-up directory. You can change the directory if required.

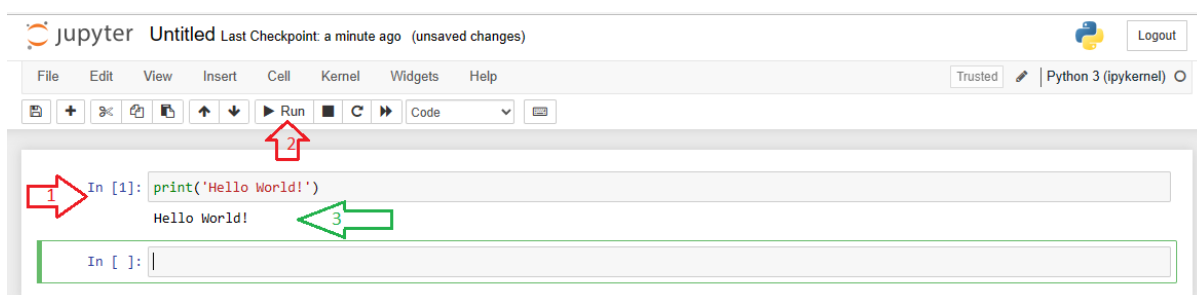
- Browse to the folder in which you would like to create your first notebook (e.g., I navigated to my Lab folder under my AUT's OneDrive), click the "New" drop-down button in the top-right and select "Python 3":



- Your first Jupyter Notebook (.jpynb) will open in new tab:



- Type `print('Hello World!')` in the code/input cell (In []) and hit the Run button:

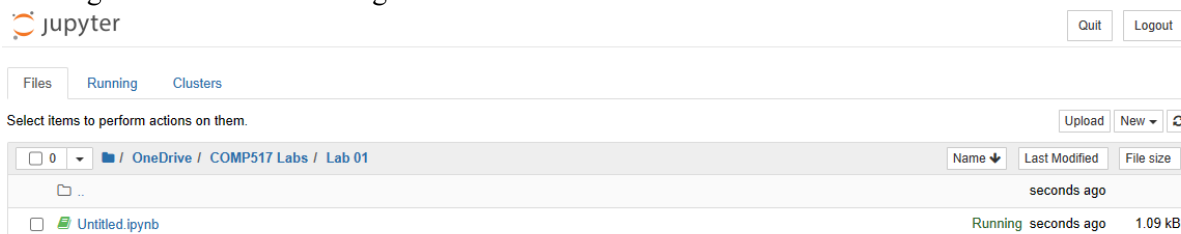


You just managed to **execute** your first code from the browser and **display** the output! You can edit your code in the browser with **proper indentation**. Change the above code to:

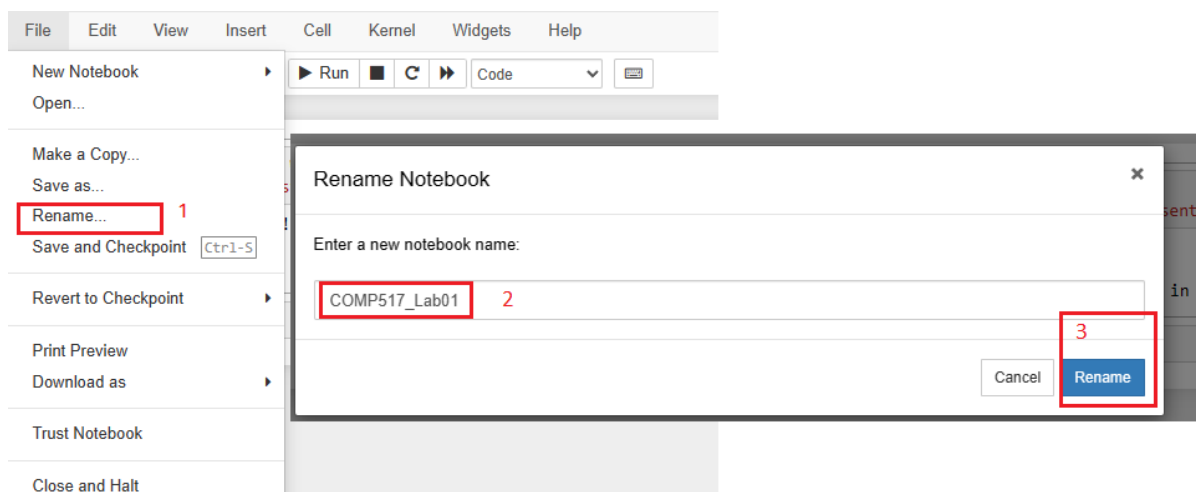
```
print('Hello World!!\n')
print(r'I just added extra ! to my previous code and used \n
to print this sentence in new line!')
```

*#Question: What happens if you remove the **r** from above command?*

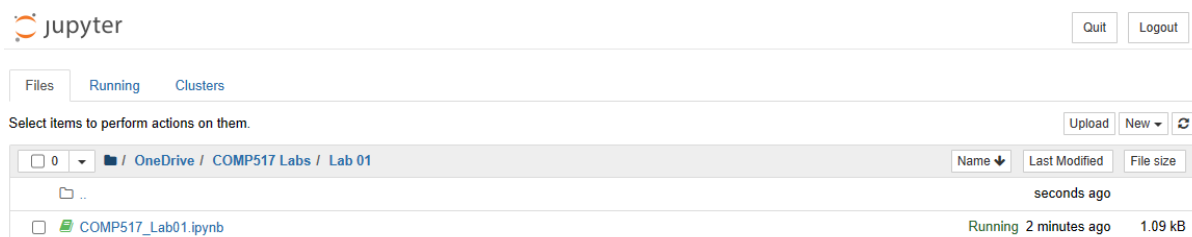
- Switch back to the dashboard (it is already opened in your browser). You will find your new Jupyter Notebook *Untitled.ipynb* created under your dashboard. Note the Running status showing the notebook is running:



- Back to your code file, from the context menu select '*Rename*' to initiate the renaming process. The file name will become editable, allowing you to modify it. Enter the new name for the notebook file and click on '*Rename*' button to save the new name.

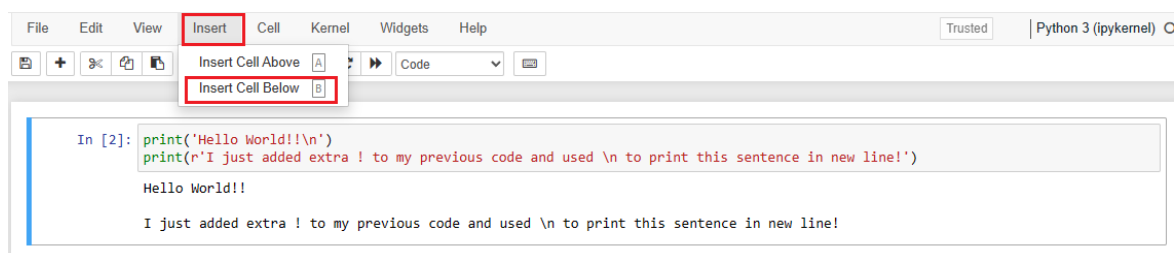


- Your notebook file name is now updated:

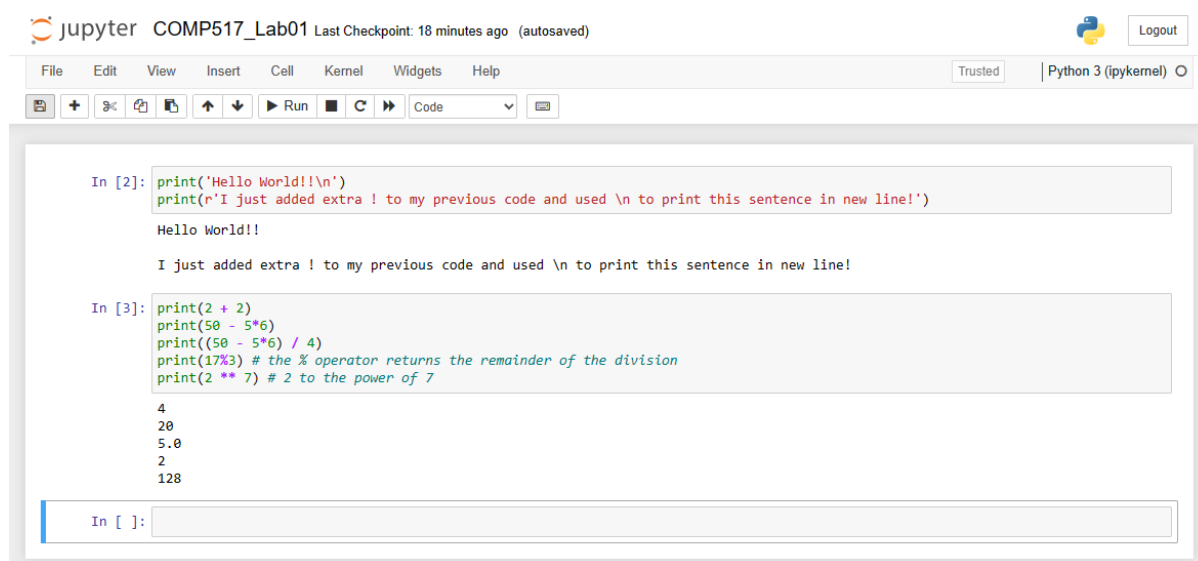


Getting Started with Coding

To add code in a Jupyter Notebook, click on the "+" button in the toolbar to add a new cell. In the cell, you can write your Python code and run the code by hitting the Run button. You can also use the Insert button to add the new cell above or below the existing cell. (Note that the Run button only runs the code within a cell)



1. Basic Arithmetic Operations



2. Assignment of Values to Variables

```
width = 20
height = 5 * 9
print (width * height)
# You can make the output more meaningful
print('Result=', width * height)

# Assigning values with arithmetic operations
length = 10
width = length + 5
area = length * width
print('Area=', area)

# Assigning values using different data types
name = 'John Doe' # String
age = 25 # Integer
height = 1.75 # Floating-point number
is_student = True # Boolean

# Assigning values from user input
user_input = input("Enter a weight for John: ")
weight = int(user_input)
print('John weights ', weight)

#Question: What is the difference between double quotation "" and single quotation ' '?

# Assigning values from user input using &
name, age = input('Enter name and age (separated by &):').split('&')
print("Name:", name)
print("Age:", age)

#Question: What does the split('&') do in the above code? What do you get if you remove it?
```

```
# Assigning values from a dictionary
```

```
person = {  
    "name": "Ella Marija Lani Yelich-O'Connor",  
    "age": 26,  
    "Known as": "Lord",  
    "occupation": "Singer/songwriter"  
}
```

```
person_name = person["name"]
```

```
person_age = person.get("age", 0)
```

```
#Question: What is the '0' in person.get? Is it necessary? Why?
```

```
# Assigning values with conditional statements
```

```
precipitation = 12
```

```
weather = 'Sunny' if precipitation <= 0 else 'Cloudy'
```

```
print('It is '+weather+ ' today. The total rainfall depth so far is  
'+str(precipitation)+' (mm)!')
```

```
# Assigning values with multiple assignment
```

```
x = y = z = 10
```

```
a, b, c = 1, 2, 3
```

```
# Assigning values with unpacking
```

```
values = [10, 20, 30]
```

```
x, y, z = values
```

String Operations

```
print('spam eggs') # single quotes
print(r'C:\some\name') # note the r before the quote
print(3 * 'un' + 'ium')

#Embed numbers within a string output
name = 'John Doe'
age = 25
height = 1.759
output = '{} is {} years old, and his height is {:.3f} meters.'.format(name, age, height)
print(output)

# Using f-strings (Python 3.6+)
output = f'{name} is {age} years old, and his height is {height:.2f} meters.'
print(output)
```

Strings can be indexed (subscripted), with the first character having index 0. There is no separate character type; a character is simply a string of size one. Indices may also be negative numbers, to start counting from the right:

```
my_string = "able was I ere I saw elba"
# Accessing individual characters
print(my_string[0])      # Output: a
print(my_string[5])      # Output: w

# Negative indexing (counting from the end of the string)
print(my_string[-1])     # Output: a
print(my_string[-8])     # Output: s
```

The colon (:) is used to specify slicing ranges. Omitting the start index means the slice starts from the beginning of the list, and omitting the end index means the slice continues until the end of the list.

```
# Slicing a substring
print(my_string[9:14])    # Output: I ere
print(my_string[:5])      # Output: able
print(my_string[11:])     # Output: ere I saw elba
print(my_string[:])       # Output: able was I ere I saw elba

# Skipping characters with step
print(my_string[::2])     # Output: al a  r  a la

# Reversing a string
print(my_string[::-1])    # Output: able was I ere I saw elba
```


3. Lists

Like strings (and all other built-in sequence types), lists can be indexed and sliced.

```
my_list = [1, 2, 3, 4, 5]

# Accessing individual elements
print(my_list[0])      # Output: 1
print(my_list[3])      # Output: 4

# Negative indexing (counting from the end of the list)
print(my_list[-1])     # Output: 5
print(my_list[-3])     # Output: 3

# Slicing a sublist
print(my_list[1:4])     # Output: [2, 3, 4]
print(my_list[:3])     # Output: [1, 2, 3]
print(my_list[2:])     # Output: [3, 4, 5]
print(my_list[:])      # Output: [1, 2, 3, 4, 5]

# Skipping elements with step
print(my_list[::2])     # Output: [1, 3, 5]

# Reversing a list
print(my_list[::-1])    # Output: [5, 4, 3, 2, 1]
```

Lists also support operations like concatenation:

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
# Concatenation: Combining two lists
concatenated_list = list1 + list2
print(concatenated_list)  # Output: [1, 2, 3, 4, 5, 6]

# Repetition: Repeating a list
repeated_list = list1 * 3
print(repeated_list)      # Output: [1, 2, 3, 1, 2, 3, 1, 2, 3]
# Length of a list: Number of elements
length = len(list1)
print(length)             # Output: 3
```

Unlike strings, which are immutable, lists are a mutable type, i.e., it is possible to change its element at a specific index:

```
# Creating a list
fruits = ['apple', '2', 'orange', 'mango']
# Changing an element at a specific index
fruits[1] = 'grape'
print(fruits)             # Output: ['apple', 'grape', 'orange', 'mango']
```

You can also add new items at the end of the list, by using the `append()` method

```
# Appending elements to the end of the list
my_list.append(6)
print(my_list)            # Output: [1, 2, 3, 4, 5, 6]
fruits.append('strawberry')
print(fruits)             #Output: ['apple', 'grape', 'orange', 'mango', 'strawberry']
```

The `insert()` method allows you to insert an element at a specific index.

```
# Inserting an element at a specific index
my_list.insert(3, 20)
print(my_list)          # Output: [1, 2, 3, 20, 4, 5, 6]
```

Note that when you use the `my_list.insert(3, 20)` statement, it inserts the value 20 at index 3, which means it will be inserted before the element that currently occupies that index. The elements with indexes greater than or equal to the insertion point will shift to the right to accommodate the new element.

The `remove()` method is used to remove an element from the list by its value:

```
# Removing an element by value
my_list.remove(3)
print(my_list)          # Output: [1, 2, 20, 4, 5, 6]
```

The **del** statement combined with the index is used to remove an element from the list by its index.

```
# Removing an element by index
del my_list[0]
print(my_list)          # Output: [2, 20, 4, 5, 6]
```

```
# Checking if an element exists in a list
print('grapes' in fruits) # Output: False
```

The `sorted()` method can be used to sort the list items in order (ascending order by default).

```
# Sort the fruits list in descending order
Sorted_fruits=sorted(fruits,reverse=True)
print("Sorted Fruits (Descending):", Sorted_fruits )
```

You can iterate over each element of a list and replace the elements using if-else statement:

```
my_list = [10, 20, -3.18, 30, -120, 50]
modified_list = [x if x > 0 else 999 for x in my_list]
print(modified_list)      # output: [10, 20, 999, 30, 999, 50]
```

A new list called `modified_list` is created using a list comprehension. The list comprehension iterates over each element `x` in `my_list`.

- If `x` is greater than 0, it is included as is in the `modified_list`.
- If `x` is not greater than 0 (i.e., it is a negative number or zero), it is replaced with 999 in the `modified_list`.

```
my_list = [1, 'apple', 3.14, 'banana', 42]
modified_list = [x if isinstance(x, (int, float)) else ' ' for
x in my_list]
print(modified_list) # output: [1, ' ', 3.14, ' ', 42]
```

The above code modifies the given list by checking the type of each element. The list contains five elements of different types: an integer (1), a string ('apple'), a float (3.14), another string ('banana'), and another integer (42).

The expression `x if isinstance(x, (int, float)) else ' '` is used as a conditional expression within the list comprehension. It checks if the element `x` is an instance of either *int* or *float* using the `isinstance()` function.

- If `x` is an integer or a float, it is included as is in the `modified_list`.
- If `x` is not an integer or a float (i.e., it is a string or some other type), it is replaced with a space (' ') in the `modified_list`.

Study below codes and provide the outputs.

```
# List Membership
my_list = [10, 20, 30, 40, 50]
membership = 40 in my_list
print(membership)      #output?

# List Count
my_list = [1, 2, 2, 3, 2, 4, 2, 5]
count = my_list.count(2)
print(count)           #output?

# List Index
my_list = ['apple', 'banana', 'cherry', 'date']
index = my_list.index('cherry')
print(index)           #output?

# List Slicing
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
sliced_list = my_list[2:7]

# The slicing operation extracts a portion of the list from
index 2 up to, but not including, index 7.
print(sliced_list)     #output?
```

Embrace the keyboard and wield your coding powers with confidence. Write the code, understand its secrets, and become the master of your own programming destiny. May the code be with you!

Jedi Mind Training:

Writing code by hand is like wielding a lightsaber—it trains your Jedi mind to understand the code's logic and syntax. Embrace the Force and feel the code flow through you!



Code Chameleon Abilities:

Writing code from scratch makes you a code chameleon—able to adapt and customize your creations. Don't settle for copy-paste monotony; let your imagination run wild and make the code dance to your tune!

There IS a chameleon in the house, but it's so slick, it changed colors and disappeared into the background!

Bug Ninja Skills:

When code misbehaves, you become a bug-slaying ninja! By writing the code yourself, you gain the power to track down errors and fix them with finesse. Unleash your debugging skills and conquer the coding realm!



Super Memory Boost:

Writing code by hand is like doing mental push-ups—it strengthens your memory muscles. As you jot down code, you're flexing your brain and locking those programming concepts in your mind fortress. Remember, practice makes perfect!



Q Dear Google

